BCS501 • MODULE 1

Software Engineering

Software, Process Models & Agile Development

Prepared By,
Dr.Manjunath H,
Assisatent Professor, CSE, BIT

Module 1 — Syllabus Roadmap

1 Software & Software Engineering

Nature of software • WebApps • SE • process • practice • myths

2 Process Models

Generic process • assessment • waterfall • incremental • evolutionary • concurrent • specialized • UP • PSP/TSP

3 Agile Development

Agility • cost of change • agile process • XP • other agile models • agile toolset

CHAPTER 1

Software and Software Engineering

What software is, why it is different, and why an engineering approach is needed.

The Nature of Software

Software is both product and vehicle

As a product, software delivers computing capability and information. As a vehicle, it controls hardware, enables communication, and supports creation of other programs.

Software as an information transformer

It can produce, manage, acquire, modify, display, or transmit information—from simple data to complex multimedia.

Why software engineering became necessary

Software systems became more sophisticated and complex.

Large teams replaced the lone programmer for many projects.

Projects still face questions about cost, schedule, errors, maintenance, and measuring progress.

Defining Software

A software product contains three essential elements:

Programs

Instructions that provide desired features, functions and performance.

Data structures

Structures that enable programs to manipulate information.

Descriptive information

Hard-copy or virtual information describing operation and use.

Key idea: software is a logical rather than a physical system element.

Three Distinctive Characteristics of Software

01

Engineered, not manufactured

Quality is concentrated in engineering and design; software projects cannot be managed like manufacturing projects.

02

Does not wear out

Software does not deteriorate from physical wear, but changes can introduce defects and cause deterioration.

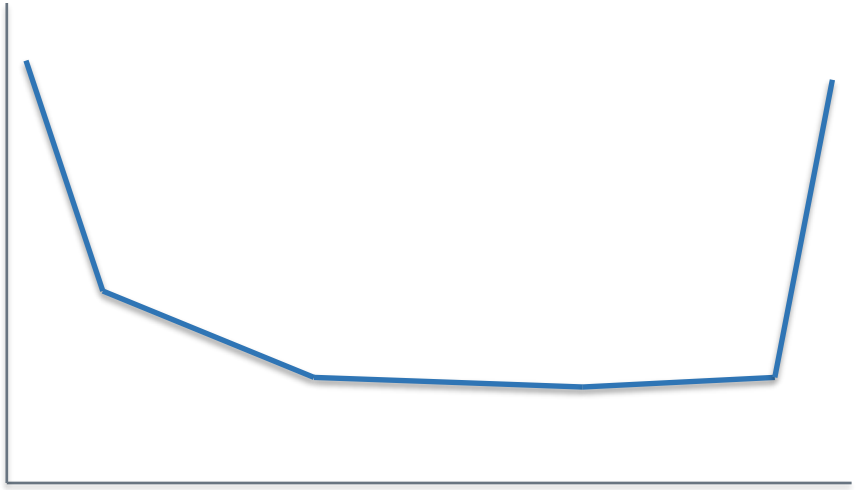
03

Mostly custom-built

Although component-based construction is growing, much software is still built for specific needs.

Software Does Not Wear Out — It Deteriorates

Hardware: typical bathtub curve



Physical wear-out raises failure rate later.

Software: change-driven deterioration



Changes can introduce side effects and raise the baseline failure rate.

Software maintenance is complex because there are no “software spare parts.”

Software Application Domains

System software

Serves other programs; includes OS components, drivers and utilities.

Application software

Solves a specific business or technical need.

Engineering / scientific

Uses computational algorithms for scientific and engineering work.

Embedded software

Resides in a product/system and controls features and functions.

Product-line software

Provides a capability for many different customers.

Web applications

Network-centric applications ranging from simple pages to sophisticated systems.

Artificial intelligence

Uses nonnumerical algorithms for complex problems.

Legacy Software

Legacy systems are older programs that have been continually modified to meet changing business requirements and computing platforms.

Longevity

May have been developed decades ago.

Business criticality

Often supports core business functions and may be indispensable.

Maintenance burden

Can be costly to maintain and risky to evolve.

Quality concerns

May have poor documentation, convoluted code or weak change history.

Legacy evolution is driven by new environments, new business requirements, interoperability, or re-architecting.

The Unique Nature of WebApps

WebApps are network-centric software applications delivered through web technologies and browsers. They combine software functionality with content and presentation.

- Network intensiveness
- Concurrency
- Unpredictable load
- Performance
- Availability
- Data driven
- Content sensitive
- Continuous evolution
- Immediacy
- Security
- Aesthetics

WebApp Characteristics — Why They Matter

Concurrency + load

Many users may access at once; usage can vary greatly.

Performance + availability

Users expect responsive systems and often 24/7/365 access.

Content + aesthetics

Information quality and look-and-feel affect perceived quality.

Continuous evolution + immediacy

WebApps can change continuously and often face short time-to-market windows.

Security

Network access makes protection of data and infrastructure essential.

Pressman’s point: WebApps almost always exhibit the full combination of these characteristics.

What Is Software Engineering?

Software should be engineered because modern systems are complex, widely relied upon, and expected to evolve.

Understand the problem

Stakeholders have different needs; requirements must be understood before a solution is built.

Design is pivotal

Complex systems require careful attention to interactions among elements.

Quality matters

Software failure can range from inconvenience to catastrophic consequences.

Maintainability matters

Valuable software tends to live longer and must accommodate change.

Definitions of Software Engineering

Bauer — seminal definition

Software engineering is the establishment and use of sound engineering principles to obtain economically software that is reliable and works efficiently on real machines.

IEEE definition

The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; and the study of such approaches.

Pressman emphasizes both discipline and adaptability: we need an engineering approach that can remain agile.

Software Engineering as a Layered Technology

Tools	Automated / semi-automated support
Methods	Technical how-to for communication, analysis, design, construction, testing and support
Process	Framework that organizes work, work products, milestones, quality and change
Quality focus	Organizational commitment to quality and continuous improvement

The Software Process

A process is a collection of activities, actions and tasks performed to create a work product.

Activity

Broad objective, such as communication with stakeholders.

Action

A set of tasks that produces a major work product.

Task

A small, well-defined objective with a tangible outcome.

A software process is not a rigid prescription; it is an adaptable approach selected for the project, team and context.

Five Generic Framework Activities

1

Communication

Understand stakeholders and gather requirements.

2

Planning

Define tasks, risks, resources, work products and schedule.

3

Modeling

Build models to understand requirements and design.

4

Construction

Generate code and perform the testing needed to uncover errors.

5

Deployment

Deliver software/increment, obtain evaluation and feedback.

The same framework can support small programs, WebApps and large complex systems; the task details are adapted.

Umbrella Activities

Software project tracking and control

Applied throughout the software process to manage progress, quality, change, risk or work products.

Risk management

Applied throughout the software process to manage progress, quality, change, risk or work products.

Software quality assurance

Applied throughout the software process to manage progress, quality, change, risk or work products.

Technical reviews

Applied throughout the software process to manage progress, quality, change, risk or work products.

Measurement

Applied throughout the software process to manage progress, quality, change, risk or work products.

Software configuration management

Applied throughout the software process to manage progress, quality, change, risk or work products.

Reusability management

Applied throughout the software process to manage progress, quality, change, risk or work products.

Work product preparation and production

Applied throughout the software process to manage progress, quality, change, risk or work products.

Software Process Adaptation

A process adopted for one project may differ significantly from another.

- Overall flow and interdependencies
- Degree of task definition
- Required work products
- Quality assurance approach
- Project tracking and control
- Detail and rigor
- Customer/stakeholder involvement
- Team autonomy and role structure

Software Engineering Practice — The Essence

1

Understand the problem

Communication + analysis

2

Plan the solution

Modeling + software design

3

Carry out the plan

Code generation

4

Examine the result

Testing + quality assurance

Pressman uses this common-sense problem-solving sequence as the essence of software engineering practice.

General Principles — 7 Rules for Good Practice

1. The reason it all exists — provide value to users.

2. KISS — keep the design as simple as possible, but no simpler.

3. Maintain the vision — preserve conceptual and architectural integrity.

4. What you produce, others will consume — design for users, developers and maintainers.

5. Be open to the future — build for change and longevity.

6. Plan ahead for reuse — reuse can reduce time, effort and cost.

7. Think! — deliberate thought before action improves results and learning.

Software Myths

Software myths are erroneous beliefs about software and the process used to build it. They sound reasonable but can lead to bad decisions.

Management myths

- "Standards and procedures alone give developers everything they need."
- "Add more programmers to a late project and catch up."
- "Outsource and then relax."

Customer myths

- "A general statement of objectives is enough to start coding."
- "Software is flexible, so requirement changes are easy at any time."

Practitioner myths

- "Once the program works, the job is done."
- "Quality cannot be assessed until the program runs."
- "Only the working program is a deliverable."
- "Software engineering means unnecessary documentation."

Myth → Reality: What the Chapter Emphasizes

MYTH	REALITY
More people always speed up a late project	Adding people creates communication and training overhead; additions must be planned.
Requirements can be left vague	Effective and continuous customer–developer communication is needed to clarify requirements.
Working code is the only product	Models, documents, plans and other work products support engineering and future support.
Engineering means bureaucracy	The goal is quality; better quality can reduce rework and improve delivery time.

CHAPTER 2

Process Models

How software work is organized, assessed, adapted and executed through different process flows.

What Is a Software Process Model?

A process model defines how activities, actions, tasks and work products are related and how work flows through time.

Software Process — Overall framework

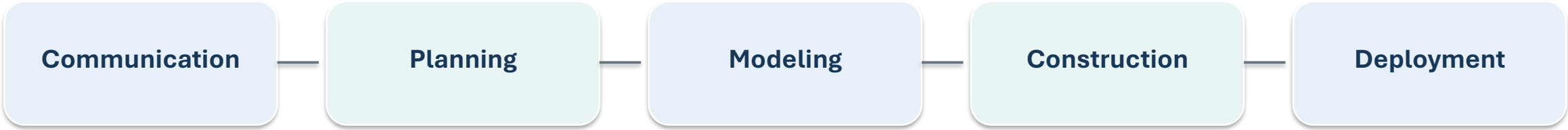
Framework Activities — Communication • Planning • Modeling • Construction • Deployment

Actions — Major engineering work within an activity

Task Set — Tasks • work products • QA points • milestones

Generic Process Model

Five framework activities + umbrella activities + process flow



Task set contains:

work tasks

work products

quality assurance points

project milestones

Umbrella activities:

tracking & control

risk management

quality assurance

configuration management

technical reviews

Process Flow Types

Linear



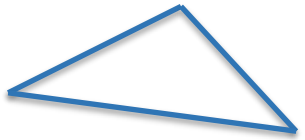
Activities execute in sequence.

Iterative



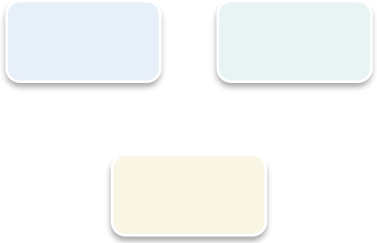
One or more activities repeat before moving on.

Evolutionary



Activities execute in a circular manner; each circuit produces a more complete version.

Parallel / Concurrent



Activities can execute in parallel with other activities.

A process pattern describes a recurring process problem, its context and a proven solution. Patterns can be combined to build a process suited to a project.

Pattern structure

A pattern identifies the problem, context and solution. A consistent template makes patterns easier to describe and reuse.

Stage pattern

Addresses a problem associated with a framework activity and can incorporate several task patterns.

Task pattern

Addresses a problem associated with a software engineering action or work task, such as requirements gathering.

Phase pattern

Defines a sequence of framework activities, even when the overall process is iterative; examples include prototyping and the Spiral model.

Process Assessment and Improvement

Assessment asks: What is the current state of the software process, and how can it be improved?

SCAMPI

Five phases: initiating → diagnosing → establishing → acting → learning.

CBA IPI

Diagnostic technique for assessing relative maturity using the SEI CMM as a basis.

SPICE

Software Process Improvement and Capability dEtermination; assessment framework aligned with ISO 15504/ISO 12207.

Bootstrap

Uses software engineering best practices to assess the “as is” process and derive maturity/improvement guidance.

Prescriptive Process Models

Prescriptive models prescribe process elements, work flow, work products, quality assurance and change-control mechanisms.

Waterfall

Sequential / linear

Incremental

Repeated linear sequences that deliver increments

Evolutionary

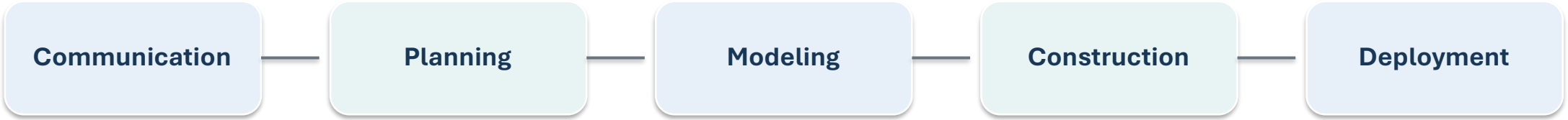
Iterative growth of increasingly complete versions

Concurrent

Activities exist concurrently in different states

Waterfall Model

Best fit when requirements are well understood and reasonably stable.



Strengths

Simple and systematic

Predictable work flow when requirements are stable

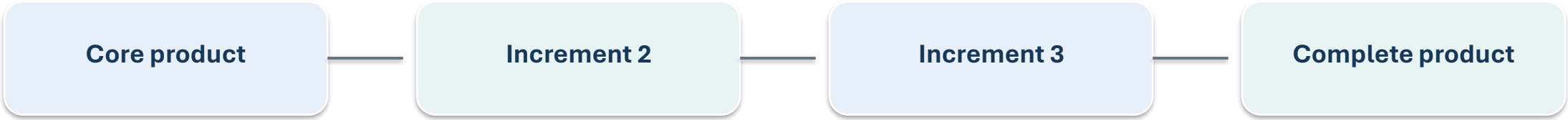
Limitations

Rigid when requirements change

Feedback/change may arrive late

Incremental Process Model

The model delivers a series of operational releases called increments; each adds functionality.



Why use it?

Operational capability reaches the customer early.

Useful when the business deadline is too tight for the full system.

Customer evaluation of early increments can guide later increments.

Increments can help manage staffing and technical risks.

Evolutionary Process Models

Evolutionary models are iterative and produce increasingly more complete versions of software.

Prototyping

Useful when requirements are fuzzy or when developers are uncertain about an algorithm, operating environment or human–machine interaction.

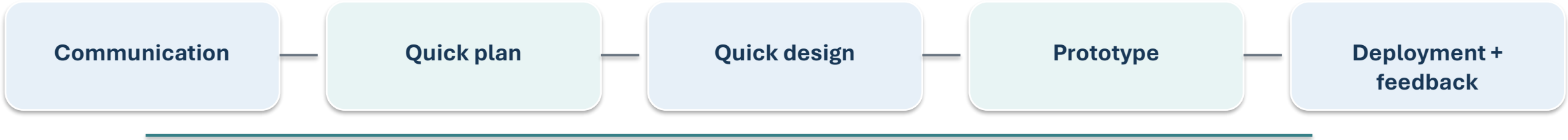
Feedback from stakeholders helps refine requirements.

Spiral model

Combines the iterative nature of prototyping with the systematic aspects of waterfall.

Risk is considered on each revolution and the product grows through evolutionary releases.

Prototyping Paradigm



Benefits

Makes the intended system tangible

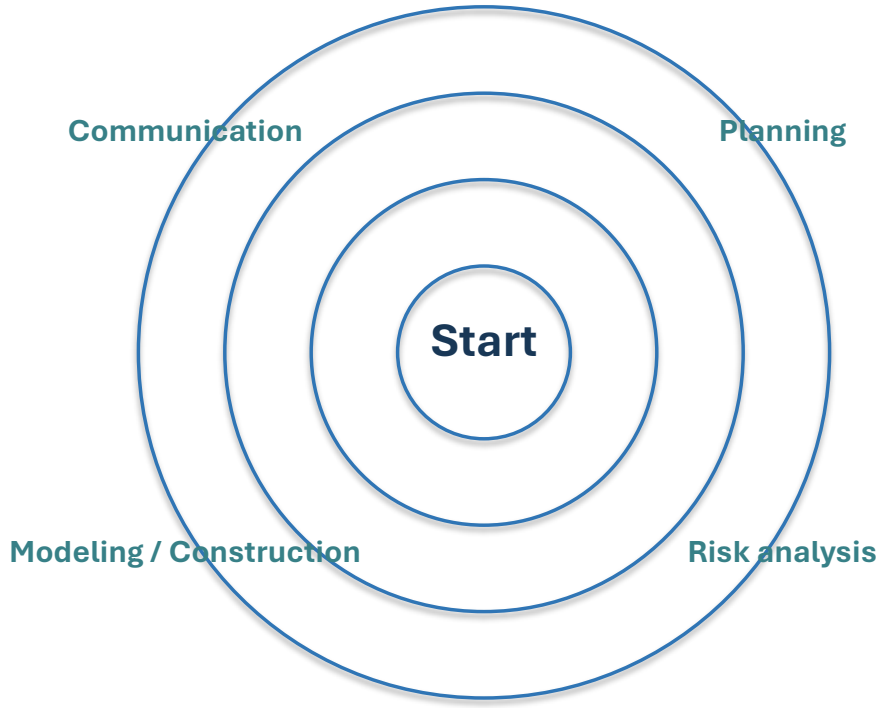
Helps discover and refine unclear requirements

Risks

Stakeholders may mistake prototype quality for production quality

Quick compromises can become permanent

Spiral Model



Key idea: risk-driven evolutionary development

Each revolution considers risk and produces a more complete release.

Anchor-point milestones help ensure stakeholder commitment.

Can continue from concept development through maintenance.

Well suited to large-scale systems where risk management is critical.

Limitation: success depends strongly on effective risk assessment and expertise.

Concurrent Development Model

Activities, actions and tasks can exist simultaneously in different states; events trigger transitions.

- **Inactive**
- **Under development**
- **Under review**
- **Baselined**
- **Awaiting changes**
- **Done**

Result: a process network that represents the current state of multiple engineering activities.

A Final Word on Evolutionary Processes

Evolutionary models address continual change, tight timelines and customer satisfaction, but they also introduce management challenges.

Planning uncertainty

The number of evolutionary cycles may be uncertain, making conventional linear planning and estimation less suitable.

Speed can become a problem

Evolution that is too fast can lead to chaos; evolution that is too slow can reduce productivity.

Flexibility vs. quality

Evolutionary development can emphasize flexibility, extensibility and speed, but teams must balance these against essential software quality.

The key challenge

Balance flexibility, development speed, software quality and customer satisfaction.

Specialized Process Models

Component-Based Development

Construct applications from prepackaged components with defined interfaces.

Formal Methods Model

Uses rigorous mathematical notation for specification, development and verification.

Aspect-Oriented Development

Localizes crosscutting concerns such as security, synchronization or persistence.

Specialized models combine process characteristics with a narrowly defined engineering objective.

Component-Based Development



Main benefit: reuse can reduce development cycle time and project cost.

Formal Methods Model

Core idea

Use rigorous mathematical notation to specify, develop and verify a computer-based system.

Why it helps

Mathematical analysis can expose ambiguity, incompleteness and inconsistency; formal design can support program verification.

Why is it not mainstream?

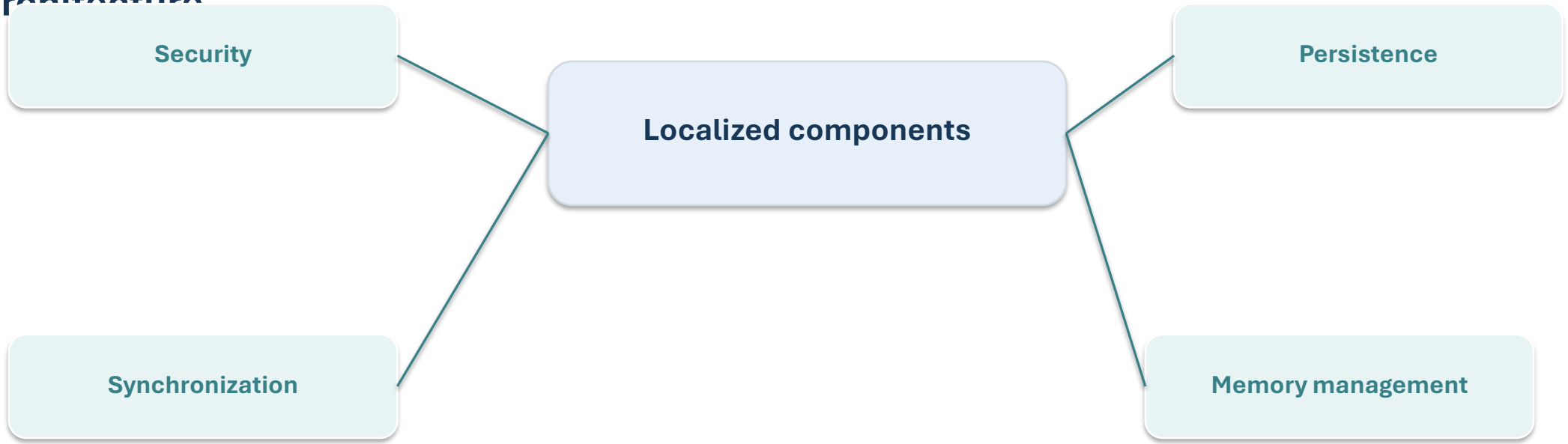
Formal models can be time-consuming and expensive.

Developers may require extensive training.

The notation can be difficult for technically unsophisticated customers to use as a communication mechanism.

Aspect-Oriented Software Development

Crosscutting concerns affect multiple functions, features or information across the architecture



These spanning concerns are modeled as aspects.

Unified Process (UP)

UP is a use-case-driven, architecture-centric, iterative and incremental process framework.

Use-case driven

Customer view of the system is captured through use cases.

Architecture-centric

Architecture receives early and continuing attention.

Iterative + incremental

The system grows through repeated increments.

Adaptable

The process should be tailored to the project’s needs.

The Unified Process grew from efforts to unify object-oriented analysis, design and modeling approaches and provide a process framework around UML.

Early 1990s

James Rumbaugh, Grady Booch and Ivar Jacobson worked toward a unified method combining strengths of their object-oriented approaches.

UML emerges

Their work produced UML, a robust modeling language. By 1997, UML had become a de facto industry standard for object-oriented development.

From UML to UP

UML provided modeling technology, but teams also needed a process framework to guide its use. Jacobson, Rumbaugh and Booch developed the Unified Process.

UP today

UP is an iterative, incremental framework for object-oriented software engineering and should be adapted to specific project needs.

Phases of the Unified Process

Inception	Elaboration	Construction	Transition	Production
Business requirements, rough architecture, project plan	Refine use cases, architecture and risks; establish architectural baseline	Implement the software increment; unit + integration + acceptance testing	Beta testing, feedback, support information; usable release	Monitor use, support environment, evaluate defects and change requests

Important: UP phases do not have to occur strictly in sequence; construction, transition and production can overlap for different increments.

UP — What Happens in Each Phase?

Inception

Stakeholders + business requirements • preliminary use cases • rough architecture • resources, risks, schedule

Elaboration

Refined use cases • five architectural views • executable architectural baseline when needed • plan review

Construction

Complete requirements/design models • implement features • unit tests • integration tests • acceptance tests

Transition

Beta testing • user feedback • manuals/troubleshooting/install support • usable release

Production

Monitor operation • support infrastructure • evaluate defects and change requests

The team adapts tasks, subtasks and work products to project needs.

Personal Software Process (PSP)

PSP is a disciplined, metrics-based approach that emphasizes personal measurement, planning and quality control.

Planning

Requirements, size/resource estimates, defect estimate, tasks and schedule

High-level design

External specifications and component design; prototypes for uncertainty

High-level design review

Formal verification to uncover design errors; maintain metrics

Development

Refine design, generate/review/compile/test code; maintain metrics

Postmortem

Analyze collected measures and use them to improve the process

Team Software Process (TSP)

Goal

Build a self-directed project team that organizes itself to produce high-quality software.

Team characteristics

Shared goals • defined roles • quantitative project data • local standards • risk assessment • project status tracking

TSP framework activities

Project launch → High-level design → Implementation → Integration and test → Postmortem

Scripts, forms and standards guide team activities.

Measurement and analysis support continual process improvement.

CHAPTER 3

Agile Development

Responding to change through collaboration, incremental delivery, adaptability and disciplined practice.

Agile Manifesto — Four Values

Individuals & interactions

over processes and tools

Working software

over comprehensive documentation

Customer collaboration

over contract negotiation

Responding to change

over following a plan

The items on the right still have value; agile thinking places greater value on the items on the left.

What Is Agility?

Agility is the ability of a software team to respond appropriately to change while maintaining forward progress.

Embrace change

Change is treated as a normal part of software development.

Facilitate communication

Teams reduce barriers among developers, managers and business people.

Deliver rapidly

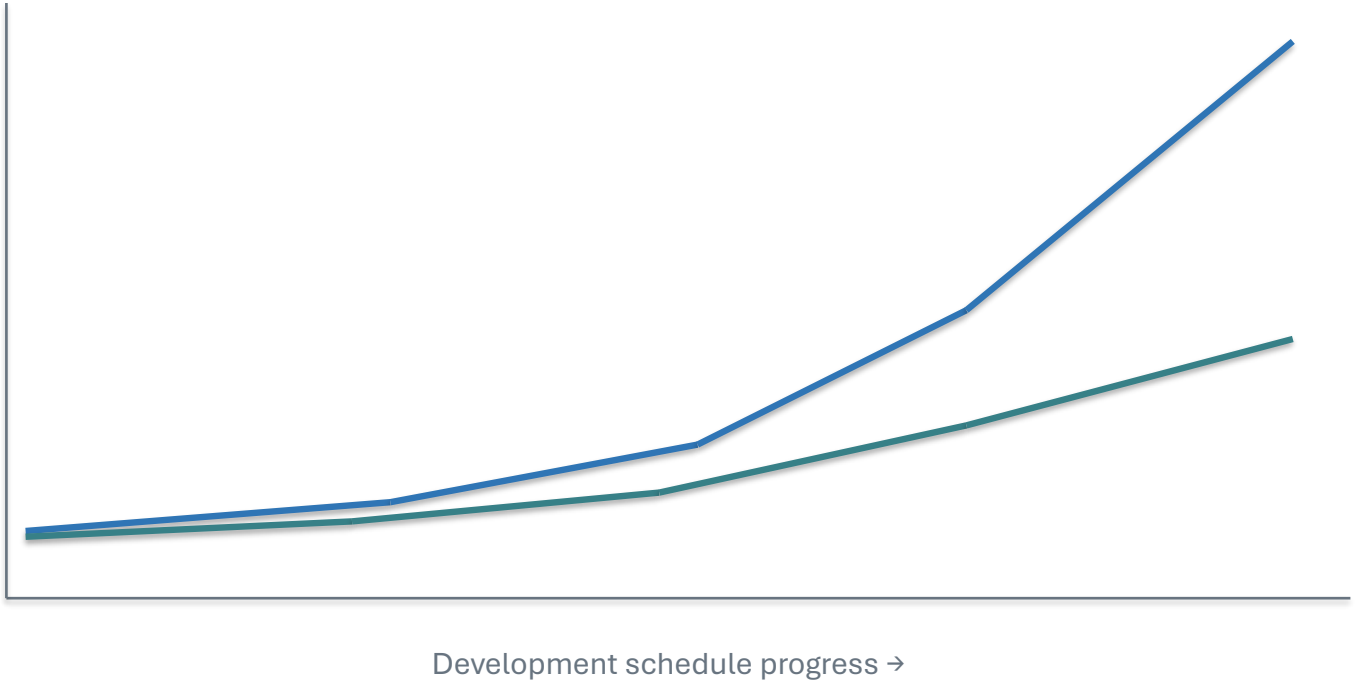
Operational software is delivered in short, useful increments.

Keep plans flexible

Planning recognizes uncertainty rather than pretending it does not exist.

Agility and the Cost of Change

Conventional view: cost of change tends to increase nonlinearly as development progresses.



Agile processes aim to “flatten” the curve by:

- incremental delivery
- continuous unit testing
- pair programming
- frequent feedback

Key idea: change can be better controlled within an increment.

What Is an Agile Process?

Requirements and customer priorities are difficult to predict in advance.

For many systems, design and construction are interleaved.

Analysis, design, construction and testing are less predictable than planning would like.

Therefore, an agile process must be adaptable, incremental and feedback-driven.

Short iterations → operational increment → customer evaluation → adaptation

12 Principles of Agile Development — 1 to 6

1. Satisfy the customer through early and continuous delivery of valuable software.

2. Welcome changing requirements, even late in development.

3. Deliver working software frequently, preferring shorter timescales.

4. Business people and developers work together daily.

5. Build around motivated individuals; provide support and trust them.

6. Face-to-face conversation is an effective way to convey information.

12 Principles of Agile Development — 7 to 12

7. Working software is the primary measure of progress.

8. Promote sustainable development at a constant pace.

9. Continuous attention to technical excellence and good design enhances agility.

10. Simplicity — maximizing work not done — is essential.

11. Best architectures, requirements and designs emerge from self-organizing teams.

12. At regular intervals, the team reflects and adjusts its behavior.

Agile development has generated strong debate about its benefits, applicability and relationship to conventional software engineering.

The debate

Some argue that traditional methods overemphasize documentation; others worry that lightweight agile approaches may struggle when

Avoid extremes

The methodology debate should not become a matter of belief. Decisions should be based on the needs and realities of the project.

What really matters

The goal is to deliver software that meets customer needs now and has the quality needed for future extension and scaling.

Pressman's conclusion

There is no single absolute answer. Agile concepts can adapt good software engineering ideas, so the best approach can draw from both perspectives.

Human Factors in Agile Teams

- Competence
- Common focus
- Collaboration
- Decision-making ability
- Fuzzy problem-solving ability
- Mutual trust & respect
- Self-organization

Core message: the process should mold to the needs of the people and team.

Extreme Programming (XP)

XP is presented by Pressman as the most widely used agile process and an example of agile practice in detail.

- **Communication**
- **Simplicity**
- **Feedback**
- **Courage**
- **Respect**

XP framework activities

Planning

Design

Coding

Testing

XP Process — Planning & Design

Planning / Planning Game

- Listen to stakeholders
- Create user stories
- Customer assigns business value / priority
- Team estimates cost
- Group stories into releases
- Use project velocity to help plan later releases

Design

- Follow Keep-It-Simple principles
- Design only for current story needs
- Use CRC cards in an object-oriented context
- Use spike solutions for difficult design problems
- Refactor to improve internal structure without changing external behavior

XP Process — Coding & Testing

Coding practices

- Create unit tests before coding
- Pair programming: two people at one workstation
- Continuous integration
- Real-time problem solving and code review
- Keep implementation focused on passing the tests

Testing practices

- Automated unit tests
- Regression testing after changes
- Integration and validation testing can occur frequently
- Acceptance tests are derived from user stories and focus on customer-visible functionality

Industrial XP (IXP) & the XP Debate

Industrial XP adds practices for larger organizations

Readiness assessment

Project community

Project chartering

Test-driven management

Retrospectives

Continuous learning

Issues in the XP debate

Requirements volatility

Conflicting customer needs

Informal requirements

Lack of formal design for some complex systems

Pressman’s conclusion: recognize weaknesses and adapt the process to organizational and project needs.

Other Agile Process Models — Overview

ASD	Adaptive Software Development
DSDM	Time-boxed incremental prototyping
FDD	Feature-driven, client-valued functions
AM	Lightweight, purposeful modeling
Scrum	Backlogs, sprints, daily Scrum, demos
Crystal	Family of agile processes for different project contexts
LSD	Lean principles adapted to software
AUP	Serial in the large; iterative in the small

Adaptive Software Development (ASD)

Speculation

Initiate the project and plan adaptive release cycles from mission, constraints and basic requirements.

Collaboration

Develop components through collaborative, self-organizing team work.

Learning

Use focus groups, technical reviews and postmortems to improve understanding.

ASD emphasizes human collaboration, self-organization and learning.

Scrum

Product backlog

Prioritized list of requirements/features providing business value.

Sprint

Time-boxed work unit used to achieve backlog requirements.

Daily Scrum

Short meeting: what was done, obstacles, what comes next.

Demo

Working increment is demonstrated and evaluated by the customer.

Scrum is especially aimed at tight timelines, changing requirements and business-critical projects.

Dynamic Systems Development Method (DSDM)



DSDM uses incremental prototyping in a controlled project environment.

Functional model iteration gathers feedback from users.

Design/build revisits prototypes so they provide operational business value.

Implementation may return to functional model iteration when changes are requested.

Crystal

Crystal is a family of agile process models designed to match different project characteristics.

Common core

Family members share core elements.

Project-specific fit

Roles, patterns, work products and practices vary.

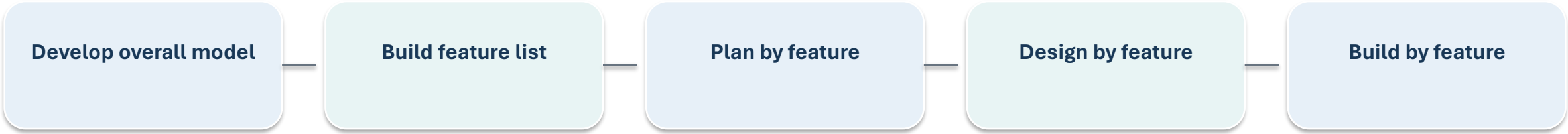
Maneuverability

Focus on delivering useful working software and preparing for the next cycle.

Choose the Crystal process that best fits the project and environment.

Feature Driven Development (FDD)

FDD decomposes the problem into client-valued features and delivers operational features incrementally.



Feature = a client-valued function that can be implemented in two weeks or less.

Quality support: inspections • SQA audits • metrics • patterns • incremental delivery

Lean Software Development (LSD)

- **Eliminate waste**
- **Build quality in**
- **Create knowledge**
- **Defer commitment**
- **Deliver fast**
- **Respect people**
- **Optimize the whole**

LSD adapts lean-manufacturing principles to software engineering.

Agile Modeling (AM)

Model with a purpose

Know why the model is being created before choosing notation/detail.

Use multiple models

Use only the small subset that provides useful insight.

Travel light

Keep models that provide long-term value; discard the rest.

Content > representation

Useful information matters more than syntactic perfection.

Know tools & models

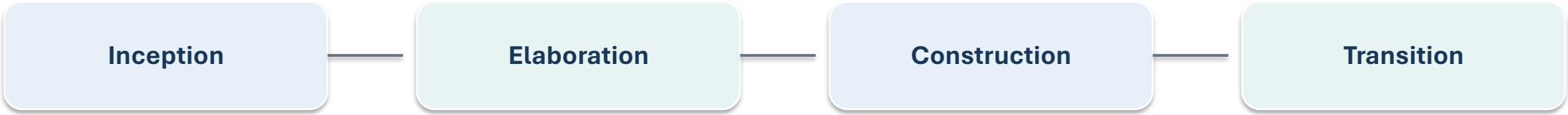
Understand strengths and weaknesses.

Adapt locally

Tune modeling to the needs of the agile team.

Agile Unified Process (AUP)

AUP uses a serial overlay for the overall project, but iterates inside each phase to deliver meaningful increments quickly.



Within iterations

Modeling — just barely good enough

Implementation • Testing • Deployment

Configuration/project management • Environment management

A Tool Set for the Agile Process

Pressman emphasizes tools that help the team maintain a rapid flow of understanding, collaboration and working software.

Social / hiring

Pair-programming assessment and team fit.

Information radiators

Visible status displays for the increment.

Process support

Pair programming, time-boxing, collaborative work areas and electronic boards.

Communication

Whiteboards, index cards, sticky notes, physical proximity.

Project management

Earned-value or test-progress views rather than relying only on Gantt charts.